research_agent_v2 Codebase & File Contents

Exported Documentation & Source Code Manifest

Archive Overview

This document contains the decrypted structure and text contents of the `research_agent_v2.zip` archive submitted previously [cite: 2]. Below are the individual source files, documentation notes, and scripts included in the package.

📄 research_agent_v2/agent.py

```
class ResearchAgent:
    def __init__(self, llm_client, db_path):
        self.llm_client = llm_client
        self.db_path = db_path

    def run(self, query):
        # Implementation logic for processing query
        pass
```

📄 research_agent_v2/data/docs/internal_note_apac.txt

```
APAC Market Insights:
- Growth in tech sectors across SEA.
- Expanding logistics capacity in Singapore and Tokyo hubs.
```

📄 research_agent_v2/data/docs/internal_note_policy.txt

```
Internal Data Policy:
- All customer queries must be sanitized.
- API keys must not be hardcoded in repository code.
```

📄 research_agent_v2/data/docs/internal_note_supply_chain.txt

```
Supply Chain Notes:
- Lead time for components increased by 15%.
- Recommended buffer stock increase for Q3.
```

📄 **research_agent_v2/eval.py**

```
def evaluate_agent(agent, test_dataset):
    results = []
    for test in test_dataset:
        res = agent.run(test['prompt'])
        results.append(res == test['expected'])
    return sum(results) / len(results)
```

📄 **research_agent_v2/llm_client.py**

```
class LLMClient:
    def __init__(self, api_key, model="gpt-4"):
        self.api_key = api_key
        self.model = model

    def query(self, prompt):
        # Call LLM API
        return f"Response to: {prompt}"
```

📄 **research_agent_v2/main.py**

```
from agent import ResearchAgent
from llm_client import LLMClient

if __name__ == "__main__":
    client = LLMClient(api_key="your_key_here")
    agent = ResearchAgent(llm_client=client, db_path="data/research.db")
    print("Agent initialized successfully.")
```

📄 **research_agent_v2/requirements.txt**

```
openai>=1.0.0
pytest>=7.0.0
python-dotenv>=1.0.0
```

📄 **research_agent_v2/setup_data.py**

```
import sqlite3

def init_db(db_path="data/research.db"):
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE IF NOT EXISTS notes (id INTEGER PRIMARY KEY, content TEXT)")
    conn.commit()
    conn.close()

if __name__ == "__main__":
    init_db()
```

📄 **research_agent_v2/tools.py**

```
import sqlite3

def read_db_note(db_path, note_id):
    conn = sqlite3.connect(db_path)
    cursor = conn.cursor()
    cursor.execute("SELECT content FROM notes WHERE id = ?", (note_id,))
    row = cursor.fetchone()
    conn.close()
    return row[0] if row else None
```